

N2-EXAM – 2026-10

SOLUCIÓN

Pregunta 1 (30 pts). Implemente en Python la función `fecha_mas_reciente`, tal y como se describe en la documentación (siguiente página). No es necesario crear aserciones.

Lista de chequeo:

- ☐ La función incluye **2** doctests significativos y no redundantes.
- ☐ La función **no** usa funciones como `min()` y `max()`.
- ☐ La función usa el operador ternario y **no usa condicionales.**
- ☐ La función usa la comparación lexicográfica explicada a continuación.

Puede asumir que las tres fechas dadas siempre están en el formato `'YYYY-MM-DD'` (Año-Mes-Día). En Python, este formato permite que las fechas puedan compararse directamente como strings, ya que el orden lexicográfico coincide con el orden cronológico. Ejemplos de comparaciones:

```
"2005-02-15" < "2006-06-10" # → True (Porque 2005 es anterior a 2006)
"2010-08-23" > "2009-12-31" # → True (Porque 2010 es posterior a 2009)
"2015-03-10" < "2015-03-20" # → True (Mismo año y mes, pero el día 10 es anterior al día 20)
```

NOTA: Al crear los doctests, tenga en cuenta que un resultado esperado del tipo string, siempre va entre comillas simples, por ejemplo:

```
>>> funcion_de_ejemplo("abc", "def", "ghi") # Comentario que describe al doctest
'abc'
```

Debe responder en la siguiente página.

Rúbrica de calificación de la Pregunta 1 (30 pts):

Descripción	Cumple	Cumple parcialmente	Incumple
Cumple con las buenas prácticas del curso (BP-1 a BP-28): • 20 pts si cumple. • 10 pts si incumple una sola. • 0 pts si incumple 2 o más.			
La implementación de la función cumple con lo solicitado: • 10 pts si cumple a cabalidad con lo solicitado. • 5 pts si el algoritmo es parcialmente correcto, falla en detalles muy menores o falla máximo un ítem de la lista de chequeo. • 0 pts si el algoritmo es parcialmente correcto, pero contiene errores graves, falla dos o más ítems de la lista de chequeo o no se presentó una solución.			
Total:			

```
def fecha_mas_reciente(fecha1: str, fecha2: str, fecha3: str) -> str:
    """
    Retorna la fecha más reciente. En caso de empate, se puede retornar cualquiera de ellas.

    Args:
        fecha1 (str): Primera fecha a comparar. Está en el formato 'YYYY-MM-DD'.
        fecha2 (str): Segunda fecha a comparar. Está en el formato 'YYYY-MM-DD'.
        fecha3 (str): Tercera fecha a comparar. Está en el formato 'YYYY-MM-DD'.

    Returns:
        (str) La fecha más reciente (cronológicamente mayor) entre las tres.
```

Ejemplo de solución (con 2 doctests extra):

```
>>> fecha_mas_reciente("2021-01-01", "2022-01-01", "2023-01-01") # Orden por año
'2023-01-01'

>>> fecha_mas_reciente("2015-05-20", "2015-12-01", "2015-11-30") # Orden por mes (mismo año)
'2015-12-01'

>>> fecha_mas_reciente("2010-06-15", "2010-06-30", "2010-06-02") # Orden por día (mismo año y mes)
'2010-06-30'

>>> fecha_mas_reciente("2019-03-10", "2019-03-10", "2018-12-31") # Empate entre 2 fechas máximas
'2019-03-10'

>>> fecha_mas_reciente("2000-01-01", "2000-01-01", "2000-01-01") # Empate entre 3 fechas máximas
'2000-01-01'
"""
return fecha1 if fecha2 <= fecha1 >= fecha3 else fecha2 if fecha1 <= fecha2 >= fecha3 else fecha3
```

- En **N2-PROY: CupiCine**, el diccionario de una película, siempre tiene las siguientes llaves:

titulo (str): Título de la película. Ejemplo: "Matrix".

genero (str): Género cinematográfico de la película. Ejemplo: "Ciencia ficción".

duracion (int): Duración de la película en minutos. Ejemplo: 136. Debe ser ≥ 0 .

pais_origen (str): País de origen de la producción. Ejemplo: "EE.UU.".

idioma_original (str): Idioma original en que fue grabada la película. Ejemplo: "Inglés".

fecha_estreno (str): Fecha de estreno en formato "YYYY-MM-DD". Ejemplo: "2025-07-31".

es_subtitulada (bool): Indica si la película tiene subtítulos. Ejemplo: True.

es_3D (bool): Indica si la película es proyectada en formato 3D. Ejemplo: False.

es_IMAX (bool): Indica si la película está disponible en formato IMAX. Ejemplo: True.

clasificacion_edad (str): Clasificación por edad. Debe ser uno de los siguientes valores:

- "G" (Apta para todo público), constante asociada: CLASIFICACION_G,
- "PG" (Mayores de 10 años o en compañía de un adulto), constante asociada: CLASIFICACION_PG,
- "PG-13" (Mayores de 13 años o 13 años con un adulto), constante asociada: CLASIFICACION_PG_13,
- "NC-17" (Prohibida para menores de 17 años), constante asociada: CLASIFICACION_NC_17,
- "R" (Permitida solo para mayores de 18 años), constante asociada: CLASIFICACION_R.

total_espectadores (int): Número de espectadores que han visto la película. Ejemplo: 5. Debe ser ≥ 0 .

num_premios (int): Número de premios que ha ganado la película. Ejemplo: 4. Debe ser ≥ 0 .

es_nominada (bool): Indica si la película ha sido nominada a algún premio. Ejemplo: True.

rating_critica (float): Calificación de la crítica en una escala decimal. Ejemplo: 8.7. Debe ser ≥ 0.0

- En **N2-PROY: CupiCine**, la siguiente es una implementación válida de `es_apta_para_edad()`:

```
def es_apta_para_edad(pelicula: dict, edad_espectador: int, va_con_adulto: bool) -> bool:
    """
    Determina si la película dada es apta para un espectador según clasificacion_edad.

    Args:
        pelicula (dict): Diccionario que representa una película.
        edad_espectador (int): Edad del espectador en años. Debe ser mayor o igual a 0.
        va_con_adulto (bool): Indica si el espectador va acompañado de un adulto.

    Returns:
        (bool) True si la película es apta para la edad proporcionada, False en caso contrario.
    """

    MAYOR_EDAD = 18
    EDAD_NC = 17
    EDAD_PG_13 = 13
    EDAD_PG = 10

    clasificacion = pelicula["clasificacion_edad"]
    es_apta = False

    if clasificacion == CLASIFICACION_G:
        es_apta = True
    elif edad_espectador >= MAYOR_EDAD:
        es_apta = True
    elif clasificacion == CLASIFICACION_R and edad_espectador >= MAYOR_EDAD:
        es_apta = True
    elif clasificacion == CLASIFICACION_NC_17 and edad_espectador >= EDAD_NC:
        es_apta = True
    elif clasificacion == CLASIFICACION_PG_13 and
        ( edad_espectador > EDAD_PG_13 or (edad_espectador == EDAD_PG_13 and va_con_adulto) ):
        es_apta = True
    elif clasificacion == CLASIFICACION_PG and (edad_espectador > EDAD_PG or va_con_adulto):
        es_apta = True

    return es_apta
```

Pregunta 2 (20 pts). Implemente en Python la función `pelicula_mas_antigua`, tal y como se describe en la documentación (siguiente página). No debe crear `doctests`.

Lista de chequeo:

- ☐ La función incluye **una sola aserción** que verifica el tipo de dato esperado de `p1`.
 - No es necesario verificar el tipo de otras películas, **solo de la película `p1`**.
- ☐ La función **no usa** funciones como `min()` y `max()`.
- ☐ La función usa condicionales y **no usa el operador ternario**.
- ☐ Se usa la comparación lexicográfica que aplica a fechas en el formato `'YYYY-MM-DD'`.

Se le proporcionan los siguientes doctests de ejemplo para que comprenda aún mejor el propósito de la función:

```
>>> p1 = {"titulo": "Matrix", "fecha_estreno": "1999-03-31"}
>>> p2 = {"titulo": "Titanic", "fecha_estreno": "1997-12-19"}
>>> p3 = {"titulo": "Avatar", "fecha_estreno": "2009-12-18"}
>>> p4 = {"titulo": "Gladiador", "fecha_estreno": "2000-05-05"}
>>> pelicula_mas_antigua(p1, p2, p3, p4) # Titanic es la más antigua
{'titulo': 'Titanic', 'fecha_estreno': '1997-12-19'}

>>> p1 = {"titulo": "A", "fecha_estreno": "2000-01-01"}
>>> p2 = {"titulo": "B", "fecha_estreno": "2000-01-01"}
>>> p3 = {"titulo": "C", "fecha_estreno": "2000-01-01"}
>>> p4 = {"titulo": "D", "fecha_estreno": "2000-01-01"}
>>> pelicula_mas_antigua(p1, p2, p3, p4) # Todas empatan, retorna p4 por ser la última
{'titulo': 'D', 'fecha_estreno': '2000-01-01'}

>>> p1 = {"titulo": "Uno", "fecha_estreno": "2010-01-01"}
>>> p2 = {"titulo": "Dos", "fecha_estreno": "2009-12-31"}
>>> p3 = {"titulo": "Tres", "fecha_estreno": "2009-12-31"}
>>> p4 = {"titulo": "Cuatro", "fecha_estreno": "2009-12-31"}
>>> pelicula_mas_antigua(p1, p2, p3, p4) # Empate entre p2, p3, p4, retorna p4 por ser última
{'titulo': 'Cuatro', 'fecha_estreno': '2009-12-31'}
```

Debe responder en la siguiente página.

Rúbrica de calificación de la Pregunta 2 (20 pts):

Descripción	Cumple	Cumple parcialmente	Incumple
Cumple con las buenas prácticas del curso (BP-1 a BP-28): • 10 pts si cumple. • 5 pts si incumple una sola. • 0 pts si incumple 2 o más.			
La implementación de la función cumple con lo solicitado: • 10 pts si cumple a cabalidad con lo solicitado. • 5 pts si el algoritmo es parcialmente correcto, falla en detalles muy menores o falla máximo un ítem de la lista de chequeo. • 0 pts si el algoritmo es parcialmente correcto, pero contiene errores graves, falla dos o más ítems de la lista de chequeo o no se presentó una solución.			
Total:			

```

def pelicula_mas_antigua(p1: dict, p2: dict, p3: dict, p4: dict) -> dict:
    """
    Retorna el diccionario de la película con la fecha de estreno más antigua.
    Asuma que las fechas de estreno siempre están en el formato "YYYY-MM-DD" (Año-Mes-Día).

    Args:
        p1, p2, p3, p4 (dict): Diccionarios con la información de las cuatro películas.

    Returns: (dict) Diccionario correspondiente a la película con la fecha de estreno más antigua.
        En caso de empate, se retorna la última película en el orden p1, p2, p3, p4.
    """
    assert isinstance(p1, dict), "Se esperaba p1 del tipo dict, pero se recibió: " + str(type(p1))

    fecha = p1["fecha_estreno"]
    resultado = p1

    if p2["fecha_estreno"] <= fecha:
        fecha = p2["fecha_estreno"]
        resultado = p2
    if p3["fecha_estreno"] <= fecha:
        fecha = p3["fecha_estreno"]
        resultado = p3
    if p4["fecha_estreno"] <= fecha:
        resultado = p4

    return resultado

```

Pregunta 3 (30 pts). Implemente en Python la función `calcular_afinidad_por_edad_y_formato`, tal y como se describe en la documentación (siguiente página). No debe crear *doctests*.

Lista de chequeo:

- ☐ La función incluye **una sola aserción** que verifica la condición de rango de `edad_espectador`.
 - No es necesario verificar el tipo de dato, **solo el rango** para `edad_espectador`.
- ☐ La función **no** usa funciones como `min()` y `max()`.
- ☐ La función usa condicionales y **no usa el operador ternario**.
- ☐ La función usa las tres constantes que se dan definidas.

Se le proporcionan los siguientes *doctests* de ejemplo para que comprenda aún mejor el propósito de la función:

```
>>> p1 = {"clasificacion_edad": CLASIFICACION_PG_13, "es_3D": False, "es_IMAX": True, "es_subtitulada": False}
>>> round(calcular_afinidad_por_edad_y_formato(p1, 12, True, False, True, False), 1) # Apta y 3 coincidencias
0.3

>>> p2 = {"clasificacion_edad": CLASIFICACION_G, "es_3D": True, "es_IMAX": False, "es_subtitulada": True}
>>> round(calcular_afinidad_por_edad_y_formato(p2, 8, False, True, True, True), 1) # Apta y 2 coincidencias
0.2

>>> p3 = {"clasificacion_edad": CLASIFICACION_R, "es_3D": True, "es_IMAX": True, "es_subtitulada": True}
>>> calcular_afinidad_por_edad_y_formato(p3, 17, False, True, True, True) # No apta por edad (R)
0.0
```

Debe responder en la siguiente página.

Rúbrica de calificación de la Pregunta 3 (30 pts):

Descripción	Cumple	Cumple parcialmente	Incumple
Cumple con las buenas prácticas del curso (BP-1 a BP-28): • 15 pts si cumple. • 7.5 pts si incumple una sola. • 0 pts si incumple 2 o más.			
La implementación de la función cumple con lo solicitado: • 15 pts si cumple a cabalidad con lo solicitado. • 7.5 pts si el algoritmo es parcialmente correcto, falla en detalles muy menores o falla máximo un ítem de la lista de chequeo. • 0 pts si el algoritmo es parcialmente correcto, pero contiene errores graves, falla dos o más ítems de la lista de chequeo o no se presentó una solución.			
Total:			

```

def calcular_afinidad_por_edad_y_formato(pelicula: dict,
    edad_espectador: int,
    va_con_adulto: bool,
    quiere_3D: bool,
    quiere_IMAX: bool,
    acepta_subtitulada: bool) -> float:
    """
    Retorna un puntaje de afinidad entre un espectador y una película por edad y formato. Casos:
        1. Si la película no es apta para la edad del espectador, la afinidad es 0.0.
        2. Si es apta, se suman 0.1 puntos por cada coincidencia exacta:
            - Cuando la preferencia sobre 3D coincide con si la película está en 3D.
            - Cuando la preferencia sobre IMAX coincide con si la película está disponible en IMAX.
            - Cuando la preferencia sobre subtítulos coincide con si la película es subtitulada.
    Args:
        pelicula (dict): Diccionario que representa una película.
        edad_espectador (int): Edad del espectador. Debe estar entre 7 y 100, rango inclusivo.
        va_con_adulto (bool): Indica si el espectador va acompañado de un adulto.
        quiere_3D (bool): Indica si el espectador desea el formato 3D.
        quiere_IMAX (bool): Indica si el espectador desea el formato IMAX.
        acepta_subtitulada (bool): Indica si el espectador acepta los subtítulos.

    Returns: (float) Puntaje de afinidad entre 0.0 y 0.3.
    """
    PUNTO = 0.1
    EDAD_MINIMA = 7
    EDAD_MAXIMA = 100

    assert EDAD_MINIMA <= edad_espectador <= EDAD_MAXIMA, (
        "edad_espectador fuera de rango: " + str(edad_espectador)
    )

    afinidad = 0.0

    if es_apta_para_edad(pelicula, edad_espectador, va_con_adulto):
        if quiere_3D == pelicula["es_3D"]:
            afinidad += PUNTO
        if quiere_IMAX == pelicula["es_IMAX"]:
            afinidad += PUNTO
        if acepta_subtitulada == pelicula["es_subtitulada"]:
            afinidad += PUNTO

    return afinidad

```


Pregunta 4 (20 pts). Implemente en Python la función `recomendar_por_edad_y_formato`, tal y como se describe en la documentación (siguiente página). **No** debe crear *doctests* ni aserciones. Note que, **se usan solo 2 películas** (en lugar de 4) para simplificarle la implementación y evitarle una escritura muy extensa.

Lista de chequeo:

- ☐ La función **no** usa funciones como `min()` y `max()`.
- ☐ La función usa condicionales y **no usa el operador ternario**.

Se le proporcionan los siguientes *doctests* de ejemplo para que comprenda aún mejor el propósito de la función:

```
>>> p1 = {"clasificacion_edad": CLASIFICACION_PG_13, "es_3D": False, "es_IMAX": True, "es_subtitulada": False, "rating_critica": 8.0}
>>> p2 = {"clasificacion_edad": CLASIFICACION_PG_13, "es_3D": False, "es_IMAX": True, "es_subtitulada": False, "rating_critica": 9.0}
>>> recomendar_por_edad_y_formato(p1, p2, 12, True, False, True, False) # Empate en afinidad → gana p2 por mayor rating
{'clasificacion_edad': 'PG-13', 'es_3D': False, 'es_IMAX': True, 'es_subtitulada': False, 'rating_critica': 9.0}

>>> p1 = {"clasificacion_edad": CLASIFICACION_G, "es_3D": True, "es_IMAX": False, "es_subtitulada": True, "rating_critica": 9.0}
>>> p2 = {"clasificacion_edad": CLASIFICACION_G, "es_3D": True, "es_IMAX": False, "es_subtitulada": True, "rating_critica": 9.0}
>>> recomendar_por_edad_y_formato(p1, p2, 10, False, True, False, True) # Empate total → gana p1 por orden
{'clasificacion_edad': 'G', 'es_3D': True, 'es_IMAX': False, 'es_subtitulada': True, 'rating_critica': 9.0}
```

Rúbrica de calificación de la Pregunta 4 (20 pts):

Descripción	Cumple	Cumple parcialmente	Incumple
Cumple con las buenas prácticas del curso (BP-1 a BP-28): 10 pts si cumple. 5 pts si incumple una sola. 0 pts si incumple 2 o más.			
La implementación de la función cumple con lo solicitado: 10 pts si cumple a cabalidad con lo solicitado. 5 pts si el algoritmo es parcialmente correcto, falla en detalles muy menores o falla máximo un ítem de la lista de chequeo. 0 pts si el algoritmo es parcialmente correcto, pero contiene errores graves, falla dos o más ítems de la lista de chequeo o no se presentó una solución.			
Total:			

```

def recomendar_por_edad_y_formato(p1: dict, p2: dict,
    edad_espectador: int,
    va_con_adulto: bool,
    quiere_3D: bool,
    quiere_IMAX: bool,
    acepta_subtitulada: bool) -> dict:
    """
    Retorna la película recomendada según la mayor afinidad por edad y formato. Criterios:
        1. Se elige la película con mayor afinidad.
        2. En caso de empate, gana la de mayor rating_critica.
        3. Si el empate persiste, gana siempre p1.

    Args:
        p1 (dict): Diccionario con la información de la primera película.
        p2 (dict): Diccionario con la información de la segunda película.
        edad_espectador (int): Edad del espectador.
        va_con_adulto (bool): Indica si el espectador asiste acompañado por un adulto.
        quiere_3D (bool): Indica si el espectador desea que la película sea en formato 3D.
        quiere_IMAX (bool): Indica si el espectador desea que la película sea en formato IMAX.
        acepta_subtitulada (bool): Indica si el espectador acepta películas subtituladas.

    Returns: (dict) El diccionario de la película recomendada (p1 o p2).
    """
    a1 = calcular_afinidad_por_edad_y_formato(
        p1, edad_espectador, va_con_adulto, quiere_3D, quiere_IMAX, acepta_subtitulada)

    a2 = calcular_afinidad_por_edad_y_formato(
        p2, edad_espectador, va_con_adulto, quiere_3D, quiere_IMAX, acepta_subtitulada)

    mejor = p1
    afinidad_max = a1
    rating_max = p1["rating_critica"]

    if a2 > afinidad_max or (a2 == afinidad_max and p2["rating_critica"] > rating_max):
        mejor = p2

    return mejor

```